LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework Milestone 2

2024/25 Spring

Student Name: Aashutosh Poudyal

London Met ID: 24046334

College ID: NP01NT4A240026

Assignment Due Date: Sunday, May 4, 2025

Assignment Submission Date: Saturday, May 4, 2025

Word Count: 3370

File Links:

Onedrive Drive Link:	

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

FOC_AashutoshPoudyal.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:95737339

Submission Date
May 14, 2025, 4:24 AM GMT+5:45

Download Date
May 14, 2025, 4:27 AM GMT+5:45

File Name
FOC_AashutoshPoudyal.docx

File Size
16.0 KB

30 Pages

2,186 Words

11,496 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **22** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **22** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
9% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Submitted works
islingtoncollege on 2025-05-12 **2%**

2 Submitted works
University of Derby on 2020-05-15 **1%**

3 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2024-06-17 **1%**

4 Submitted works
The University of Wolverhampton on 2024-05-22 **<1%**

5 Submitted works
University of Westminster on 2024-03-24 **<1%**

6 Submitted works
Manipal University Jaipur Online on 2025-02-12 **<1%**

7 Submitted works
Heriot-Watt University on 2024-04-29 **<1%**

Table of Contents

Table of figure

1 Introduction

This project is about creating a basic Product Wholesale System for a skincare store using Python. The main goal is to help the shop manage its product details more easily by reading data from a text file and showing it in a neat and clear way on the screen. The products include information like name, brand, quantity, price, and country of origin.

This program is written in Python, and it uses basic tools like text files and built-in data types such as lists and dictionaries to organize the product information. The work done in this milestone makes it easier to build more features in the future.

1.1 Aims and Objectives

The aim of this project is to build up a rudimentary Product Wholesale System for a skincare boutique which can handle all stock inventory of the store. The primary milestone is centered around understanding product details from a text file, arranging the data with the help of appropriate Python data structures, and showing it such that it is readable and accessible. By developing such a foundation, the project can further be used to handle customer purchases, apply discounts on products, manage restocks, and generate invoices based on the purchases made by the customers. This milestone, therefore, ensures that the program is organized, easy to scale, and mirrors the given real-world scenario in the coursework.

- To create a text file containing product information in the required format.
- To write a Python program that reads data from the text file.
- To display the product information in a structured and readable format.
- To lay the groundwork for advanced features like inventory updates and invoice generation in future milestones.

1.2 Technologies Used

- Programming Language: Python
- Editor: IDLE
- File Format: .txt for product data
- Operating System: macOS

2 Discussion and Analysis

2.1 Algorithm

Step 1: start the system

Step 2: display shop header information then goto 3

Step 3: load product data

Step 4: display main menu options

Step 5: get user input

Step 6: if user_choice=1 then goto 7
else if user_choice=2 then goto 14
else if user_choice=3 then goto 18
else print invalid choice then goto 4

Step 7: sell products - get customer details

Step 8: display available products

Step 9: get product selection

Step 10: get quantity

Step 11: validate stock

Step 12: process sale

Step 13: generate bill

if choice=yes then goto 8
else goto 4

Step 14: restock products

Step 15: get restock amount

Step 16: update inventory then goto 4

Step 17: exit system

Step 18: end the system

2.2 Flowchart

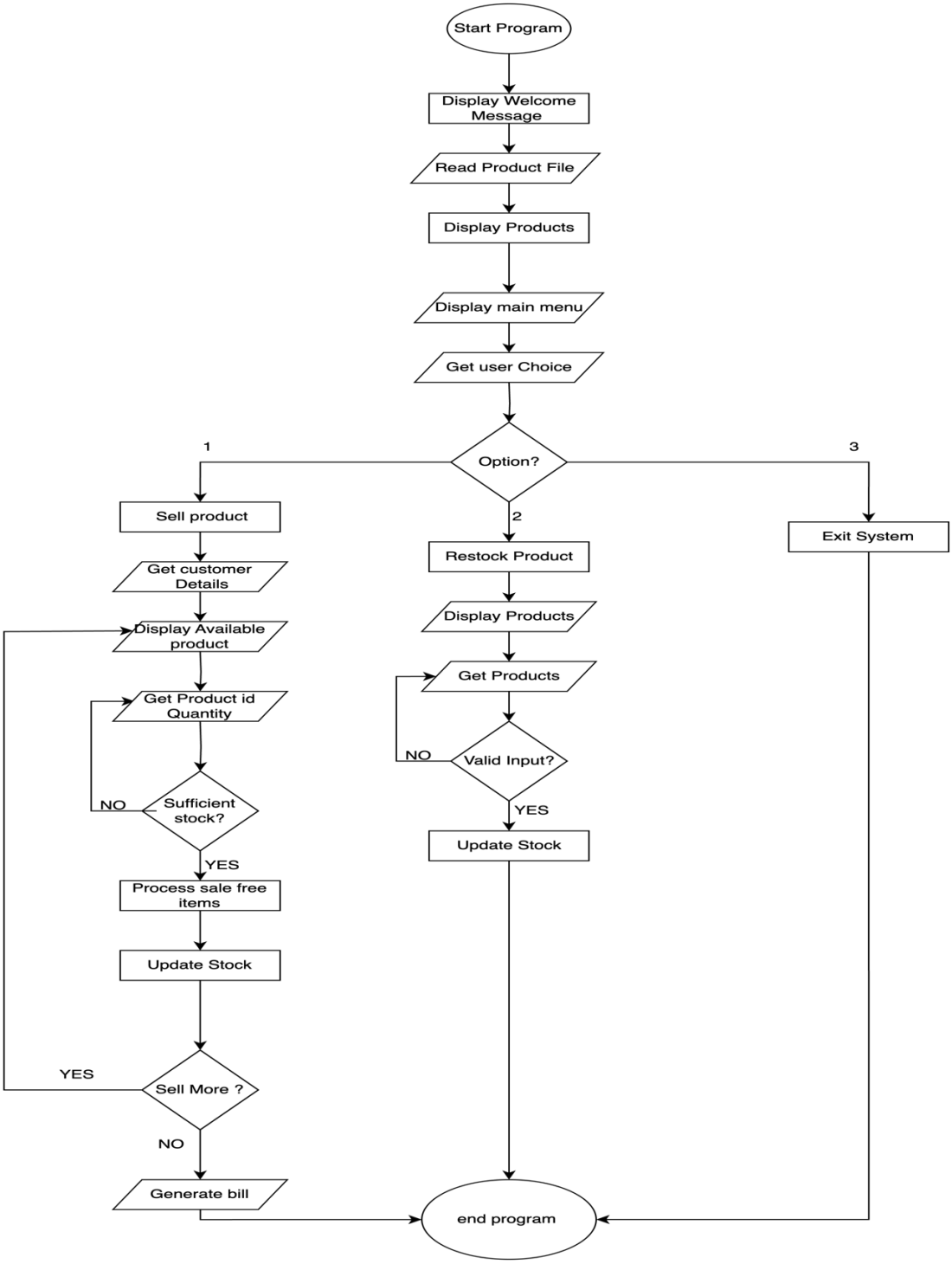

Figure 1 FlowChart

2.3 PseudoCode

1 Read Products

```
FUNCTION read_products():
    CREATE an empty dictionary d
    OPEN file "products.txt" in read mode
    READ all lines from the file into data
    SET p_id to 1
    FOR each line in data:
        REMOVE newline and split line by commas
        ADD cleaned data to dictionary d with p_id as the key
        INCREMENT p_id by 1
    RETURN d
```

2 Save Products to File

```
FUNCTION save_to_file(d):
    OPEN file "products.txt" in write mode
    FOR each product in d values:
        WRITE product details (name, company, quantity, price, country) to the file
    CLOSE the file
```

3. Sell Products

```
FUNCTION sell_products(d):
    PRINT a message for bill generation and customer details
    ASK for customer name (validate input)
    ASK for customer phone number (validate input)

    INITIALIZE sold_items as an empty list
    INITIALIZE grand_total to 0

    WHILE True:
        CALL display_products(d)
        ASK for product ID to sell (validate input)
        ASK for quantity to sell (validate input)
        GET stock quantity for the selected product

        IF stock is insufficient:
            PRINT "Insufficient stock" message and continue
        ELSE:
            CALCULATE free items for promotions (if applicable)
            UPDATE stock quantity after the sale
```

```
        CALCULATE total price for the sale and add to grand_total
        ADD sold item details to sold_items list

        ASK if more products should be sold (yes/no)
        IF response is no, BREAK from the loop

    GENERATE bill text with customer details and grand total
    CREATE a bill filename with timestamp
    PRINT the bill details to the screen
    CALL write_bill to save the bill to a file
    CALL save_to_file to update the product stock in file
```

4 Purchase Stock

```
FUNCTION purchase_stock(d):
    PRINT available products for restocking
    CALL display_products(d)

    INITIALIZE purchase_items as an empty list
    INITIALIZE grand_total to 0

    WHILE True:
        ASK for product ID to restock (validate input)
        ASK for quantity to add (validate input)
        ADD purchased quantity to stock

        CALCULATE total cost for the purchase and add to grand_total
        ADD purchase details to purchase_items list

        ASK if more products should be purchased (yes/no)
        IF response is no, BREAK from the loop

    GENERATE purchase bill text with timestamp
    CREATE a purchase bill filename with timestamp
    PRINT the purchase details to the screen
    CALL write_bill to save the purchase bill to a file
    CALL save_to_file to update the product stock in file
```

5 Main Loop

```
FUNCTION main():
    DISPLAY company information and greeting

    SET main_loop to True
    WHILE main_loop is True:
        PRINT the main menu options:
            1. Sell products to a customer
            2. Purchase stock from a manufacturer
            3. Exit the system

        ASK for option input (validate input)

        IF option is 1:
            CALL sell_products(d)
        ELSE IF option is 2:
            CALL purchase_stock(d)
        ELSE IF option is 3:
            SET main_loop to False
            PRINT "Thank you for using the system!"
        ELSE:
            PRINT "Invalid option. Please try again."

# Start the main function
CALL main()
```

2.4 Data Structures

In this project, different types of data structures are used to store and manage the product information read from the text file. Programming uses data structures to effectively manage, store, and arrange data.

1 Primitive Data Structures
The simplest data types that only store one value are called primitive data structures. They serve as the basis for more intricate structures and are built-in types that Python offers.

Types Used:

- **String (str)**: it is used to store text data like product names, brand names, and countries.

```
name = input("Customer Name: ").strip()
```

Figure 2 String

- **Integer (int)**: it is used to represent numbers like quantity and price.

```
qty = int(input("Enter quantity to sell: "))
```

Figure 3 Integer

2 Complex Data Structures
Complex data structures can store multiple values and are used to handle and organize groups of related data.

Types Used:

- **List (list)**: Used to store multiple values like product details (name, brand, quantity, price, country) for each product.

```
sold_items.append({
    'name': item_name,
    'quantity': qty,
    'free_items': free,
    'price': item_price,
    'total': total
})
```

Figure 4 list

- **Dictionary (dict)**: Used to store multiple product entries with a unique ID as the key and product details as the value (stored as a list).

```
for key in d:
    value = d[key]
    sn = str(i)
    pname = value[0]
    cname = value[1]
    qty = value[2]
    price = value[3]
    country = value[4]
```

Figure 5 Dictionary

3 Program

3.1 Overall Program Implementation

This program is a terminal-based inventory management system for a skincare wholesale business named WeCare Wholesale. It allows the admin to manage products by selling to customers, purchasing from manufacturers, and monitoring stock levels.

The admin is presented with a simple menu-driven interface:

- Option 1: Sell products to a customer
- Option 2: Purchase stock from a manufacturer
- Option 3: Exit the system

The program continues running until the admin chooses to exit. During the session, all product data is managed in memory and synchronized with a text file to ensure persistence.

File Handling

The program reads the product details from a text file named products.txt at the start. The file stores the following product details in each line:
Product Name, Company Name, Quantity, Price, Country

When the admin performs a purchase or a sale, the stock quantities are updated directly in the file so that changes persist even after the program is closed.

The program also generates transaction bills for every purchase and sale. These bills are saved as text files with unique names that include the date and time of the transaction.

Error Handling

To ensure the system does not crash due to invalid input, the program uses error handling at multiple stages.

- When asking for a menu choice, the program checks if the input is a valid number.
- When selecting a product ID, the program verifies that the ID exists.
- For quantity inputs, the program ensures the input is a valid positive number.
- If an invalid input is given (such as letters instead of numbers or negative values), the program shows a clear error message and asks the user to try again.

These checks help prevent unexpected crashes and guide the user to correct input mistakes.

3.2 Purchase and Sale Process

1. Sale Product.

a. Select option 1 from main menu

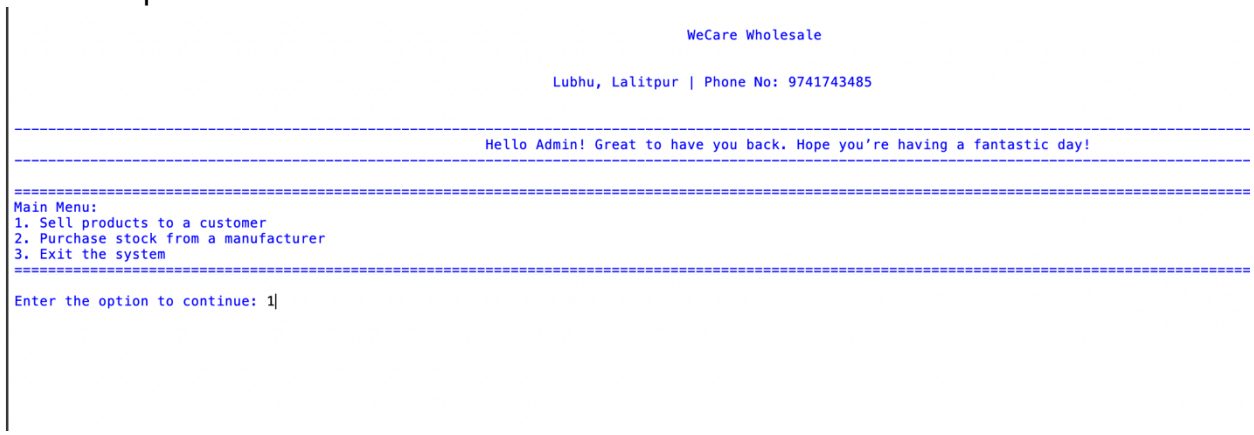

Figure 6 Select option 1 from main menu

b. Enter valid customer details

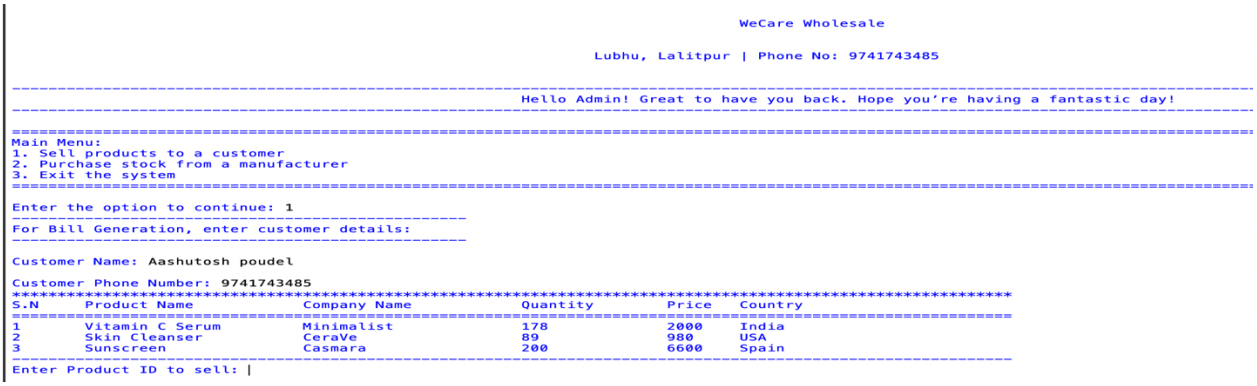

Figure 7 Enter valid customer details

c. Select products and quantities

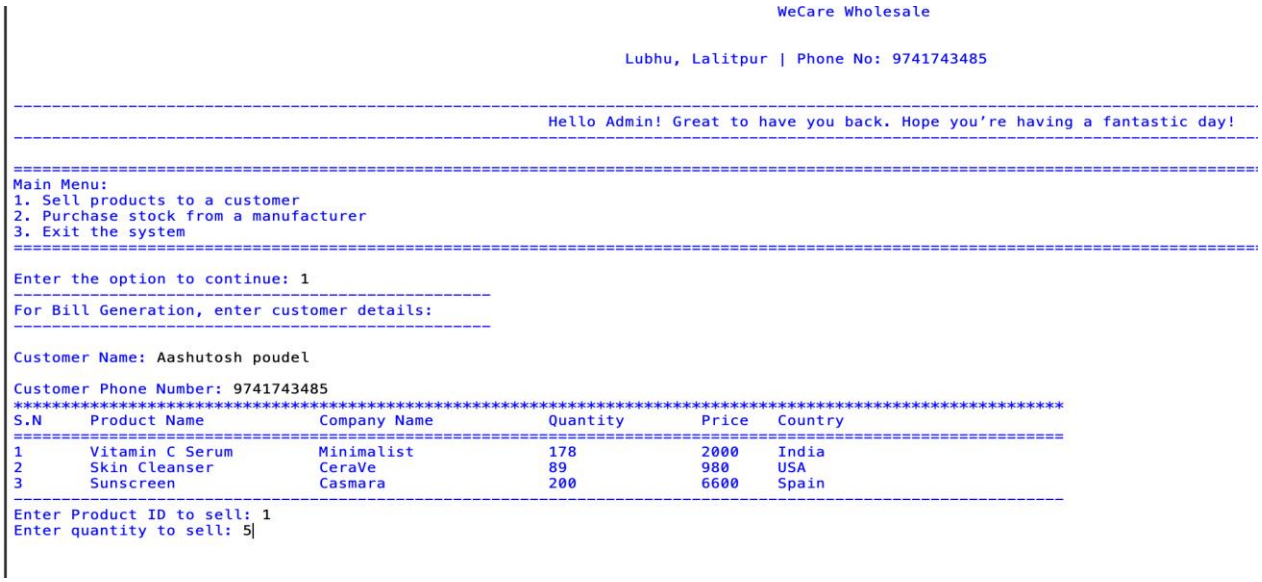

Figure 8 Select Products and Quantites

e. View sales summary with free items

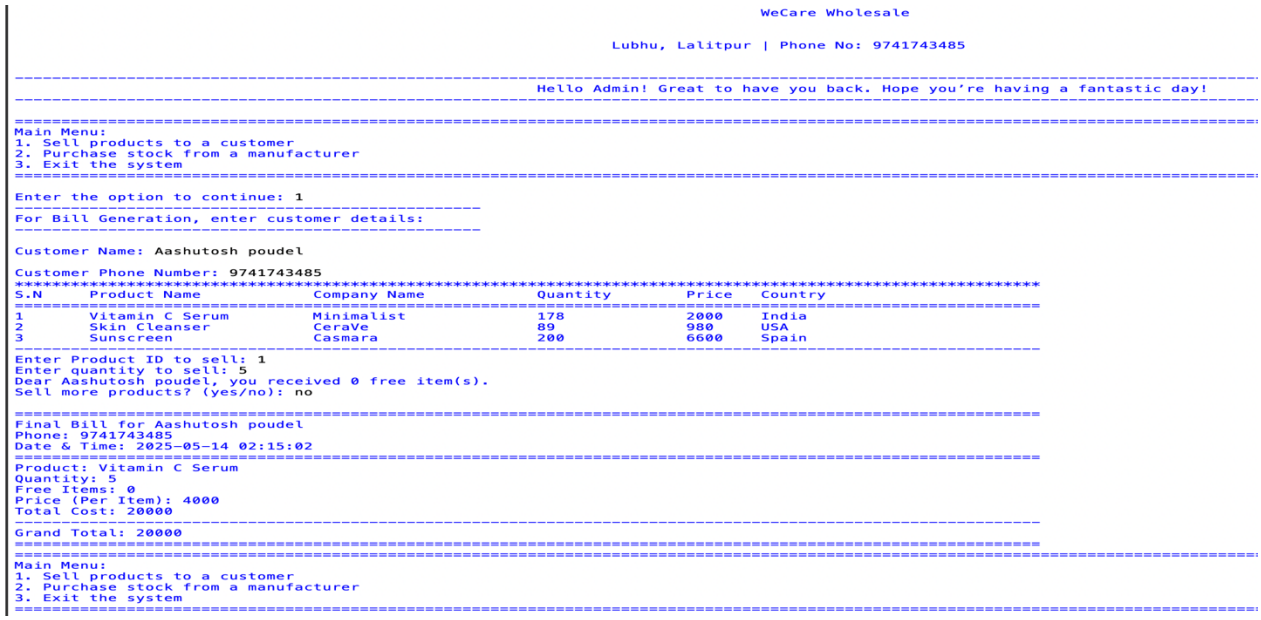

Figure 9 View sales summary with free items

2. Purchase Product.

a. Select option 2 from main menu

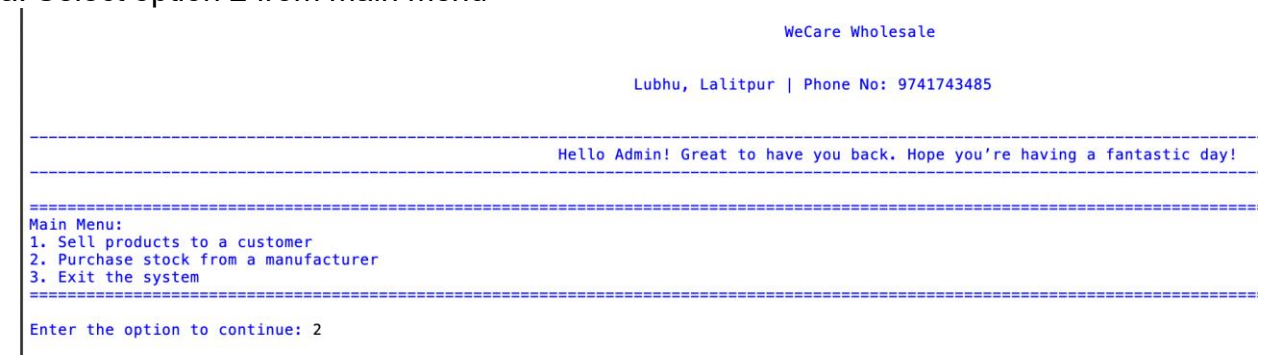

Figure 10 Select option 2 from main menu

c. Enter product ID and quantity to purchase

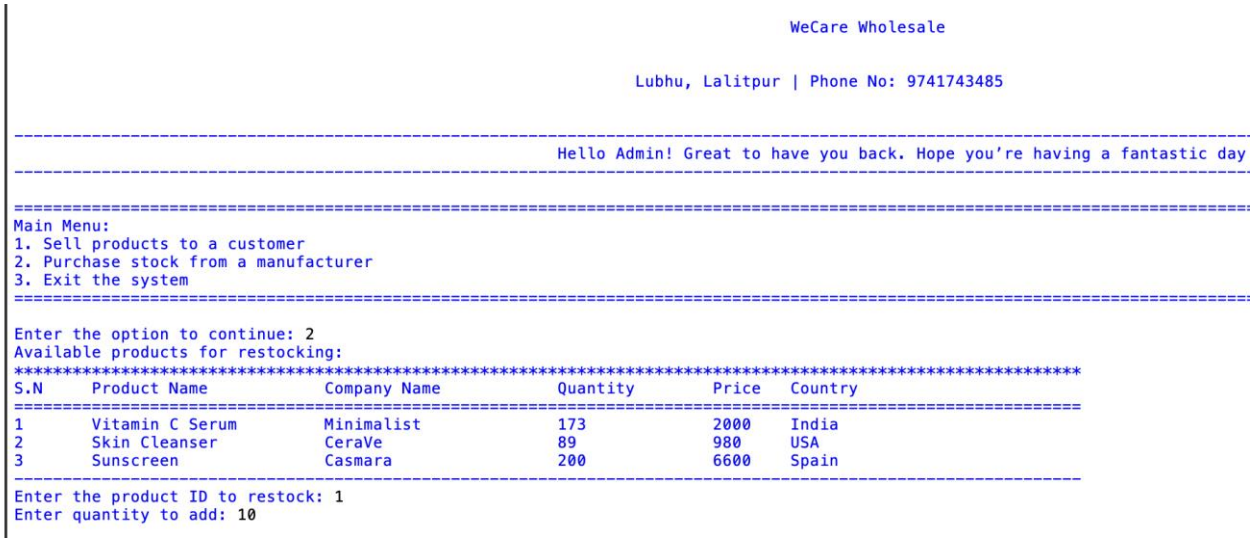

Figure 11 Enter product ID and quantity to purchase

d. Confirm additional purchases if needed

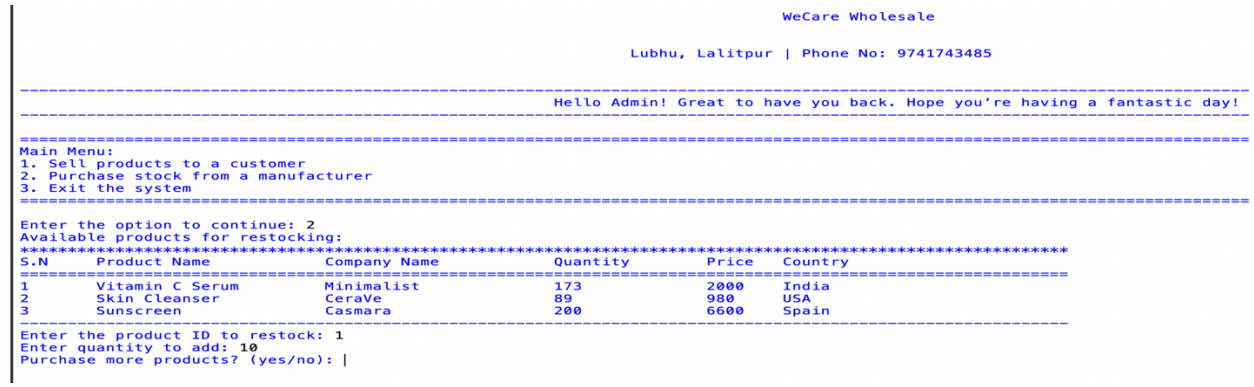

Figure 12 Confirm additional purchases if needed

e. System displays purchase summary

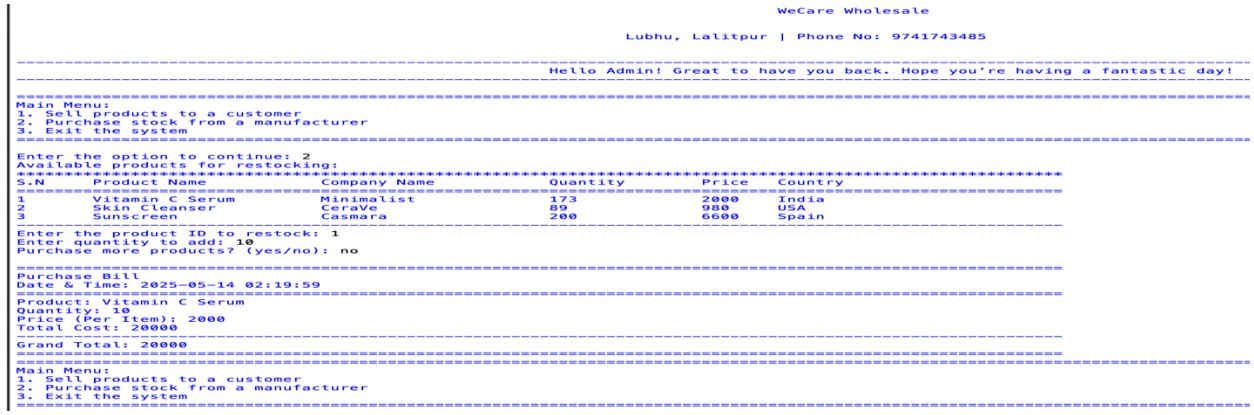

Figure 13 System displays purchase summary

3.3 Creation of txt file

The program uses file handling to manage both the product inventory and billing system

- At startup, it reads the product details from a file called products.txt. Each line in this file represents a product with its name, brand, quantity, price, and country.

- When the admin performs purchase or sale operations, the program updates this file to reflect the current stock.

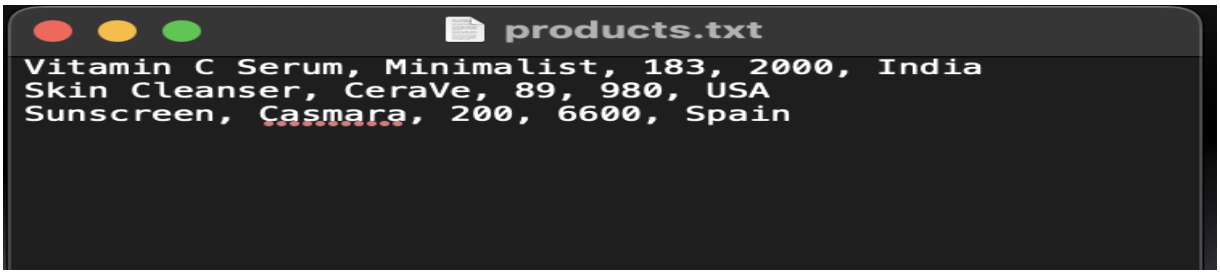

Figure 14 Creation of txt file

3.4 Opening text and show the bill

Upon starting, the program displays:

- A welcome message centered on the screen, stating the name of the business: WeCare Wholesale.
- Business address and phone number.
- A personalized message: “Hello Admin! Great to have you back. Hope you’re having a fantastic day!”

This creates a professional and user-friendly environment before proceeding with any operations.

After each sale or purchase:

- The bill is shown immediately in the console, formatted clearly.

- It includes a breakdown of each product with totals.
- Any special offers (like Buy 3 Get 1 Free) are mentioned.

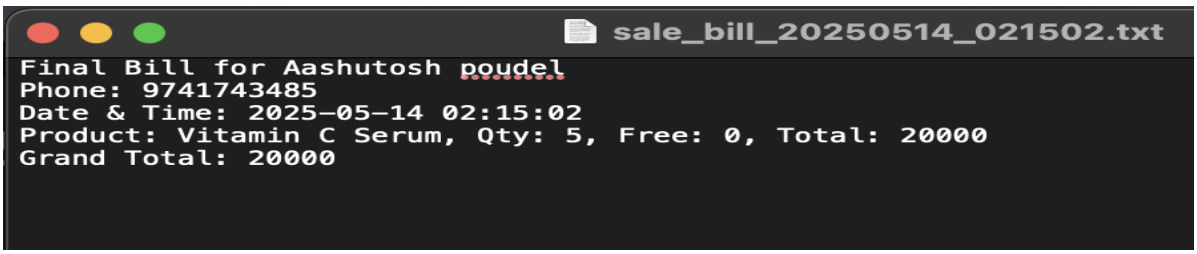

Figure 15 bill for sale

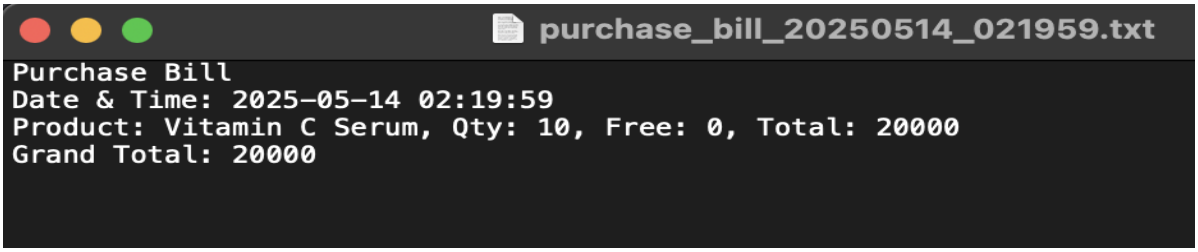

Figure 16 purchase bill

4 Testing

4.1 Test 1 – Exception Handling with try and except

Objective	Show implementation of try and except
Action	Provide invalid (non-integer) input when prompted for menu selection or quantity.
Expected Result	Program should display a message like “Invalid input. Please enter a number.” and continue running.
Actual Result	The program catches the exception and prints the appropriate error message without crashing.
Conclusion	The program successfully handles invalid inputs using try and except.

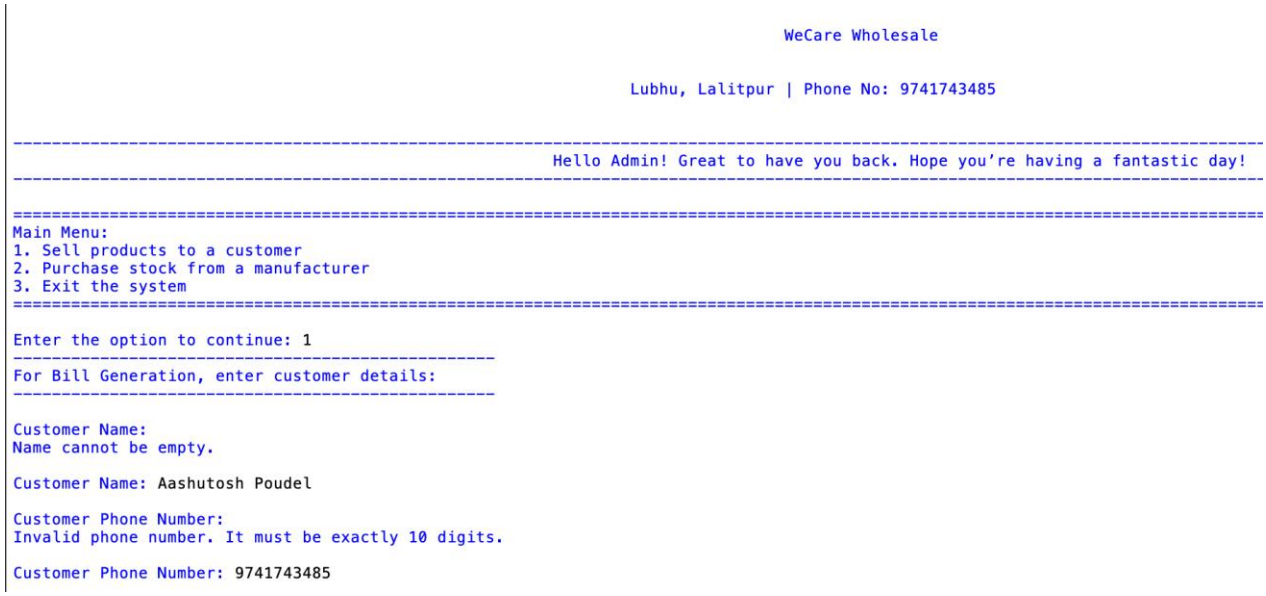

Figure 17 Test 1

4.2 Test 2 – Invalid Input During Purchase/Sale

Objective	Validate selection for purchase/sale input validation
Action	Enter a negative number or a non-existing product ID

Expected Result	System should detect invalid input and prevent the action with a message
Actual Result	System displayed: "Invalid Product ID." and "Quantity must be a positive number."
Conclusion	The system correctly handles invalid inputs during purchase/sale.

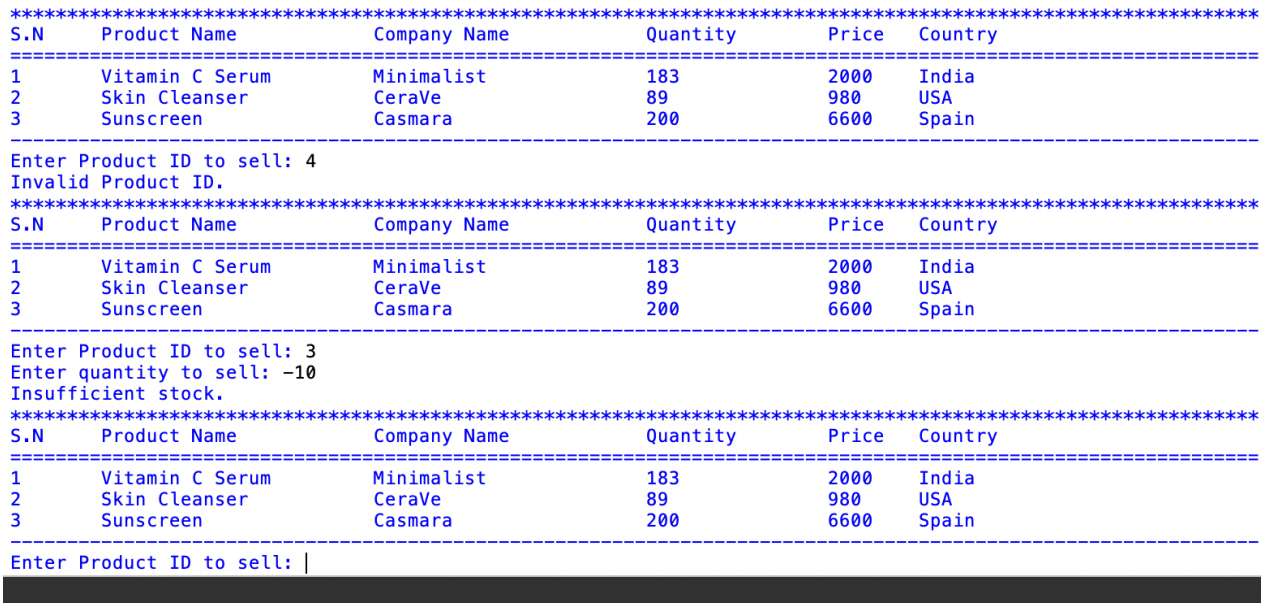

Figure 18 Test 2

4.3 Test 3 – File Generation for Purchase of Products

Objective	Validate file generation after purchasing products
Action	Purchase multiple products
Expected Result	Products should be added to stock, total cost calculated, and purchase bill saved in a .txt file
Actual Result	Products added to stock; bill file like purchase_bill_YYYYMMDD_HHMMSS.txt created with correct info
Conclusion	File generation for purchase works as intended.

```
==========================================================================================================
Main Menu:
1. Sell products to a customer
2. Purchase stock from a manufacturer
3. Exit the system
==========================================================================================================

Enter the option to continue: 2
Available products for restocking:
**********************************************************************************************************
S.N     Product Name            Company Name            Quantity        Price   Country
==========================================================================================================
1       Vitamin C Serum         Minimalist              173             2000    India
2       Skin Cleanser           CeraVe                  79              980     USA
3       Sunscreen               Casmara                 200             6600    Spain
----------------------------------------------------------------------------------------------------------
Enter the product ID to restock: 1
Enter quantity to add: 10
Purchase more products? (yes/no): yes
Enter the product ID to restock: 2
Enter quantity to add: 10
Purchase more products? (yes/no): no

==========================================================================================================
Purchase Bill
Date & Time: 2025-05-14 03:10:29
==========================================================================================================
Product: Vitamin C Serum
Quantity: 10
Price (Per Item): 2000
Total Cost: 20000
----------------------------------------------------------------------------------------------------------
Product: Skin Cleanser
Quantity: 10
Price (Per Item): 980
Total Cost: 9800
----------------------------------------------------------------------------------------------------------
Grand Total: 29800
==========================================================================================================
```

Figure 19 Test 2

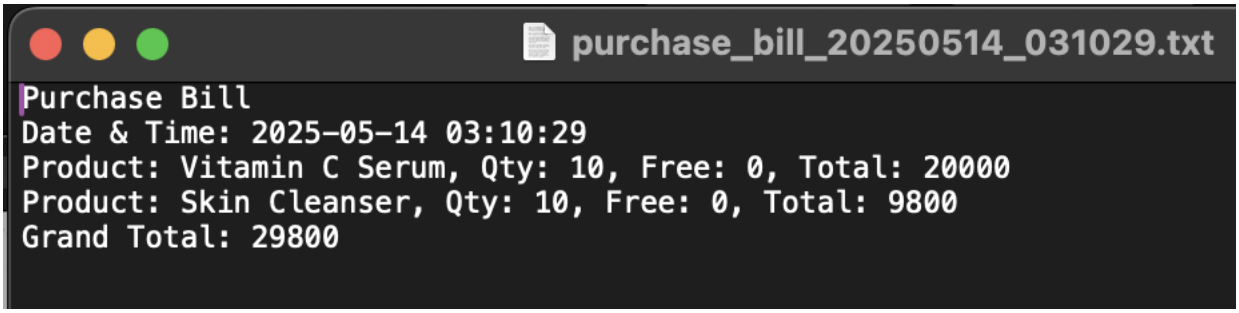

Figure 20 Test 3 bill

4.4 Test 4 – File Generation for Sale of Products

Objective	Validate file generation after selling products

Action	Sell multiple products with valid details
Expected Result	Quantity should reduce, total calculated, free item offer applied, and sales bill generated
Actual Result	Quantity updated correctly; free item message shown; sale_bill_YYYYMMDD_HHMMSS.txt file generated
Conclusion	Sales file correctly reflects multiple items with correct totals and offers.

```
--------------------------------------------------
For Bill Generation, enter customer details:
--------------------------------------------------

Customer Name: Aashu

Customer Phone Number: 1234567890
**************************************************************************************************************
S.N     Product Name            Company Name            Quantity        Price   Country
==============================================================================================================
1       Vitamin C Serum         Minimalist              183             2000    India
2       Skin Cleanser           CeraVe                  89              980     USA
3       Sunscreen               Casmara                 200             6600    Spain
--------------------------------------------------------------------------------------------------------------
Enter Product ID to sell: 1
Enter quantity to sell: 10
Dear Aashu, you received 0 free item(s).
Sell more products? (yes/no): yes
**************************************************************************************************************
S.N     Product Name            Company Name            Quantity        Price   Country
==============================================================================================================
1       Vitamin C Serum         Minimalist              173             2000    India
2       Skin Cleanser           CeraVe                  89              980     USA
3       Sunscreen               Casmara                 200             6600    Spain
--------------------------------------------------------------------------------------------------------------
Enter Product ID to sell: 2
Enter quantity to sell: 10
Dear Aashu, you received 0 free item(s).
Sell more products? (yes/no): no

==============================================================================================================
Final Bill for Aashu
Phone: 1234567890
Date & Time: 2025-05-14 03:04:45
==============================================================================================================
Product: Vitamin C Serum
Quantity: 10
Free Items: 0
Price (Per Item): 4000
Total Cost: 40000
--------------------------------------------------------------------------------------------------------------
Product: Skin Cleanser
Quantity: 10
Free Items: 0
Price (Per Item): 1960
Total Cost: 19600
--------------------------------------------------------------------------------------------------------------
Grand Total: 59600
==============================================================================================================
```

Figure 21 Test 4

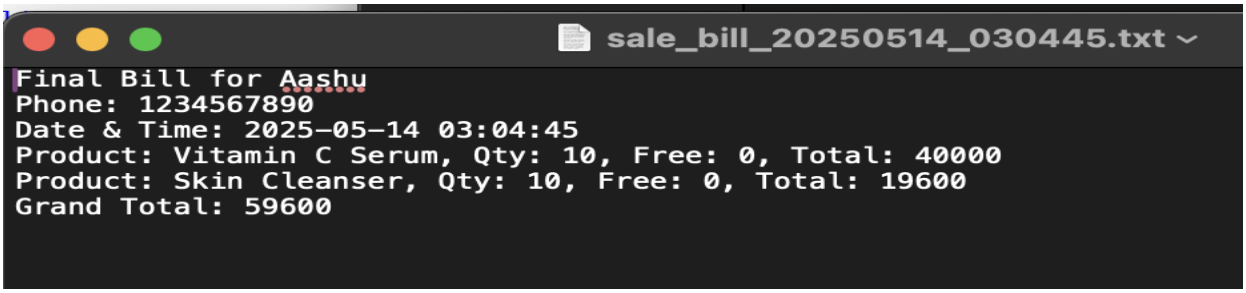

Figure 22 Test 4 bill

4.5 Test 5 – Stock Update for Products

Objective	Validate stock updates after purchase and sale
Action	Perform purchase and sale actions
Expected Result	Stock should update correctly in memory and changes should be reflected in products.txt file
Actual Result	Quantity increased after purchase and decreased after sale. products.txt updated accurately
Conclusion	

Before sale:

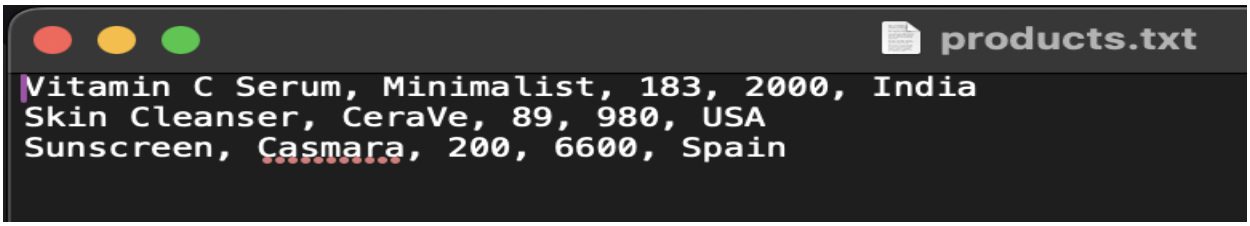

Figure 23 Test 5 before sale

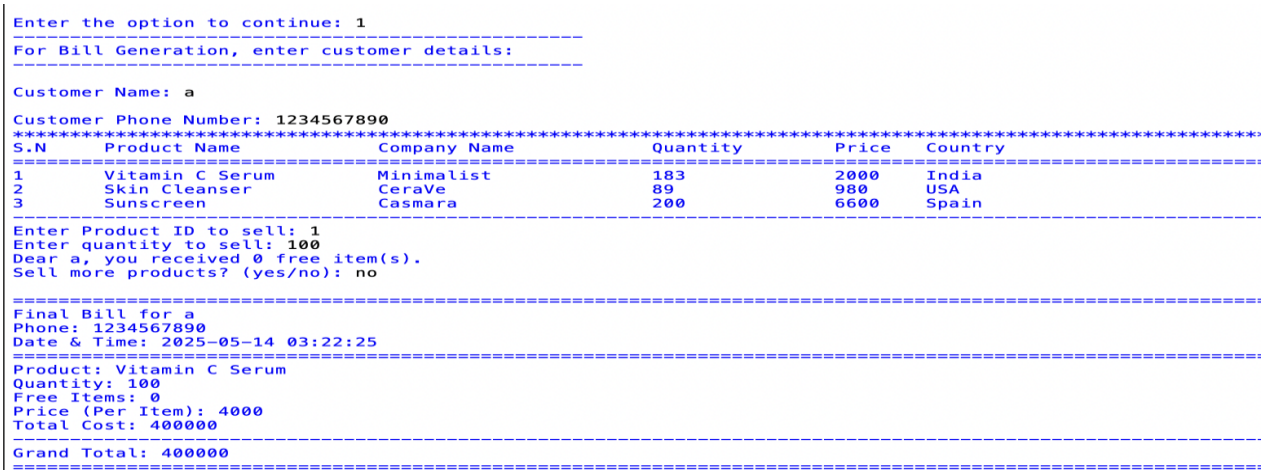

Figure 24 Test 5

After sale:

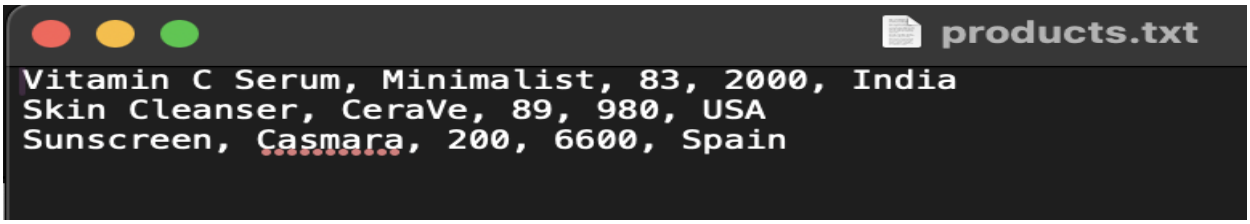

Figure 25 test 5 after sale

Before purchase:

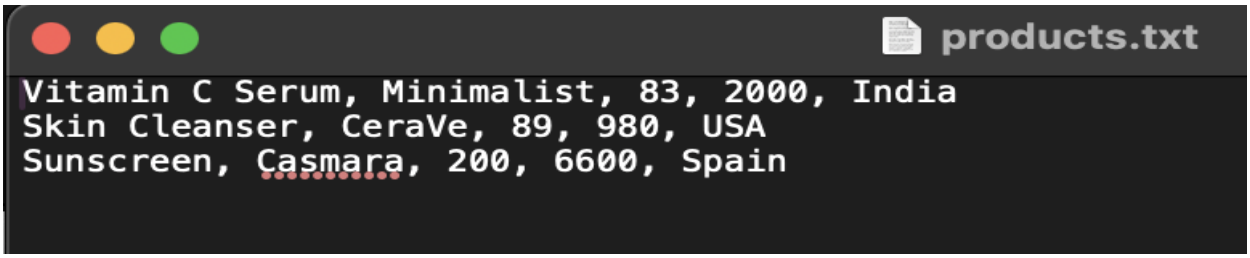

Figure 26 Test 5 before Purchase

```
Enter the option to continue: 2
Available products for restocking:
*************************************************************************************************************
S.N     Product Name            Company Name            Quantity        Price   Country
=============================================================================================================
1       Vitamin C Serum         Minimalist              83              2000    India
2       Skin Cleanser           CeraVe                  89              980     USA
3       Sunscreen               Casmara                 200             6600    Spain
-------------------------------------------------------------------------------------------------------------
Enter the product ID to restock: 1
Enter quantity to add: 100
Purchase more products? (yes/no): no

=============================================================================================================
Purchase Bill
Date & Time: 2025-05-14 03:28:23
=============================================================================================================
Product: Vitamin C Serum
Quantity: 100
Price (Per Item): 2000
Total Cost: 200000
-------------------------------------------------------------------------------------------------------------
Grand Total: 200000
=============================================================================================================
=============================================================================================================
```

Figure 27 Test 5 2nd

After purchase:

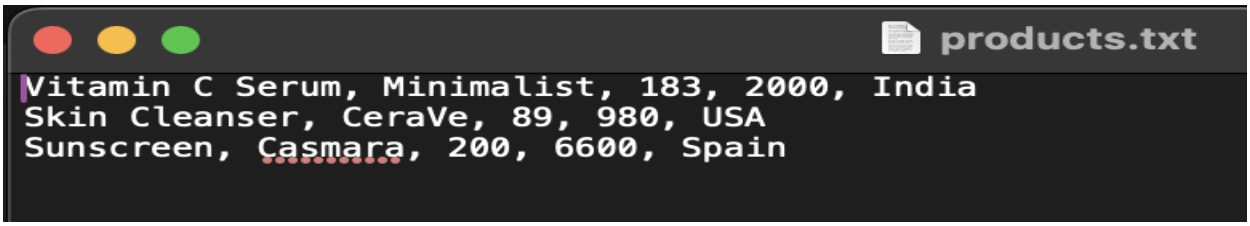

Figure 28 Test 5 After purchase

5 Conclusion

In conclusion, the WeCare Wholesale Management System was successfully created to manage the main activities of a wholesale skincare shop. The system enables the admin to record sales and purchases of products in a well-organized and simplified manner. It ensures stock quantities are updated appropriately after each transaction, and bills are generated distinctly with the right formatting for sales and purchases. Python's utilization enabled simple implementation, especially with its built-in file handling and exception handling capabilities.

Reliability was a top priority while the application was being developed, and hence it handles incorrect inputs gracefully without crashes. Features like automatic price markup for sales and the fact that promotional offers (e.g., Buy 3 Get 1 Free) are incorporated provide a real-world and functional taste to the system. In conclusion, in this project, it has been demonstrated how a simple console-based application can serve the basic needs of a wholesale company, offering efficient stock management, transaction recording, and customer service desk.

6 Appendix

READ

```
def read_products():
    d = {}
    with open("products.txt", "r") as file:
        data = file.readlines()
        p_id = 1
        for line in data:
```

```
            line = line.replace("\n", "").split(",")
            d[p_id] = line
            p_id += 1
    return d
```

WRITE

```
def save_to_file(d):
    with open("products.txt", "w") as file:
        for val in d.values():
            file.write(", ".join([val[0], val[1], str(val[2]), str(val[3]), val[4]]) + "\n")

def write_bill(filename, header, items, grand_total):
    with open(filename, "w") as bill_file:
        bill_file.write(header + "\n")
        for item in items:
            bill_file.write(f"Product: {item['name']}, Qty: {item['quantity']}, Free:
{item.get('free_items', 0)}, Total: {item['total']}\n")
        bill_file.write("Grand Total: " + str(grand_total) + "\n")
```

OPERATIONS

```
from datetime import datetime
from write import save_to_file, write_bill

def display_products(d):
    print("*" * 110)
    print("S.N    \tProduct Name        \tCompany Name        \tQuantity\tPrice   Country")
    print("=" * 110)

    i = 1
    for key in d:
        value = d[key]
        sn = str(i)
        pname = value[0]
        cname = value[1]
        qty = value[2]
        price = value[3]
        country = value[4]
```

```
        print(sn + "      \t" +
              pname + " " * (20 - len(pname)) + "\t" +
              cname + " " * (20 - len(cname)) + "\t" +
              qty + "\t\t" +
              price + "\t" +
              country)

        i += 1
    print("-" * 110)


def sell_products(d):
    print("-" * 50)
    print("For Bill Generation, enter customer details:")
    print("-" * 50 + "\n")

    # Customer Name Validation
    while True:
        name = input("Customer Name: ").strip()
        if name == "":
            print("Name cannot be empty.\n")
        elif name.isdigit():
            print("Name cannot be only numbers.\n")
        else:
            break

    print()

    # Customer Phone Validation
    while True:
        phone_number = input("Customer Phone Number: ")
        if phone_number.isdigit() and len(phone_number) == 10:
            break
        else:
            print("Invalid phone number. It must be exactly 10 digits.\n")

    sold_items = []
    grand_total = 0

    while True:
        display_products(d)
        try:
            pid = int(input("Enter Product ID to sell: "))
            if pid not in d:
```

```
            raise ValueError
        except:
            print("Invalid Product ID.")
            continue

        try:
            qty = int(input("Enter quantity to sell: "))
        except:
            print("Invalid quantity.")
            continue

        stock_qty = int(d[pid][2])
        free = 1 if qty == 3 else 0  # Only 1 free item for exactly 3 purchased
        total_deduct = qty + free

        if qty <= 0 or total_deduct > stock_qty:
            print("Insufficient stock.")
            continue

        print("Dear " + name + ", you received " + str(free) + " free item(s).")
        d[pid][2] = str(stock_qty - total_deduct)

        item_name = d[pid][0]
        item_price = int(d[pid][3]) * 2
        total = item_price * qty
        grand_total += total

        sold_items.append({
            'name': item_name,
            'quantity': qty,
            'free_items': free,
            'price': item_price,
            'total': total
        })

        more = input("Sell more products? (yes/no): ").lower()
        if more != "yes":
            break

    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    bill_text = "Final Bill for " + name + "\nPhone: " + phone_number + "\nDate & Time: " +
now
    bill_file = "sale_bill_" + datetime.now().strftime("%Y%m%d_%H%M%S") + ".txt"
```

```
    print("\n" + "=" * 110)
    print(bill_text)
    print("=" * 110)
    for item in sold_items:
        print("Product: " + item['name'])
        print("Quantity: " + str(item['quantity']))
        print("Free Items: " + str(item['free_items']))
        print("Price (Per Item): " + str(item['price']))
        print("Total Cost: " + str(item['total']))
        print("-" * 110)
    print("Grand Total: " + str(grand_total))
    print("=" * 110)

    write_bill(bill_file, bill_text, sold_items, grand_total)
    save_to_file(d)

def purchase_stock(d):
    print("Available products for restocking:")
    display_products(d)

    purchase_items = []
    grand_total = 0

    while True:
        try:
            pid = int(input("Enter the product ID to restock: "))
            if pid not in d:
                raise ValueError
        except:
            print("Invalid Product ID.")
            continue

        try:
            qty = int(input("Enter quantity to add: "))
            if qty <= 0:
                raise ValueError
        except:
            print("Quantity must be a positive number.")
            continue

        d[pid][2] = str(int(d[pid][2]) + qty)
        item_name = d[pid][0]
        item_price = int(d[pid][3])
        total = item_price * qty
        grand_total += total
```

```
        purchase_items.append({
            'name': item_name,
            'quantity': qty,
            'price': item_price,
            'total': total
        })

        more = input("Purchase more products? (yes/no): ").lower()
        if more != "yes":
            break

    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    bill_text = "Purchase Bill\nDate & Time: " + now
    bill_file = "purchase_bill_" + datetime.now().strftime("%Y%m%d_%H%M%S") + ".txt"

    print("\n" + "=" * 110)
    print(bill_text)
    print("=" * 110)
    for item in purchase_items:
        print("Product: " + item['name'])
        print("Quantity: " + str(item['quantity']))
        print("Price (Per Item): " + str(item['price']))
        print("Total Cost: " + str(item['total']))
        print("-" * 110)
    print("Grand Total: " + str(grand_total))
    print("=" * 110)

    write_bill(bill_file, bill_text, purchase_items, grand_total)
    save_to_file(d)
```

MAIN

```
from read import read_products
from operations import sell_products, purchase_stock

d = read_products()

print("\n" * 2)
print("\t" * 10 + "WeCare Wholesale")
print("\n")
print("\t" * 8 + "Lubhu, Lalitpur | Phone No: 9741743485")
print("\n")
```

```
print("-" * 200)
print("\t" * 7 + "Hello Admin! Great to have you back. Hope you're having a fantastic day!")
print("-" * 200 + "\n")

main_loop = True
while main_loop:
    print("=" * 200)
    print("Main Menu:")
    print("1. Sell products to a customer")
    print("2. Purchase stock from a manufacturer")
    print("3. Exit the system")
    print("=" * 200 + "\n")

    try:
        option = int(input("Enter the option to continue: "))
    except:
        print("Invalid input. Please enter a number.")
        continue

    if option == 1:
        sell_products(d)
    elif option == 2:
        purchase_stock(d)
    elif option == 3:
        main_loop = False
        print("Thank you for using the system. Have a good day, Admin!")
    else:
        print("Invalid option. Please try again.")
```

Bibliography